WEEK 3 · LESSON 14 OF 24 · HANDOUT

Building a CNN in PyTorch

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=2xqkSUhmmXU — open the lesson online to watch inside the course.

🎓 **Go deeper:** Computerphile: CNNs — Convolutional Neural Networks Explained — https://www.youtube.com/watch?v=py5byOOHZM8

🎓 **Go deeper:** PyData London: Introduction to Convolutional Neural Networks (TensorFlow and Keras) — https://www.youtube.com/watch?v=WIhI1W6NoZ0

Your goal in this lesson: Build and train a convolutional network in PyTorch, follow the shape of the data through every layer, and show why it beats a plain network on images.

The data: shapes

Real photo datasets are large downloads. To learn the ideas quickly, we make our own image dataset with code: 3,000 noisy 32 × 32 images of **circles, squares and triangles** at random sizes and positions. The same code works unchanged for real photos (Lesson 18).

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

def make_shapes(n, kinds=("circle", "square", "triangle"), size=32, seed=0):
    # n noisy grey images of random shapes at random places and sizes
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    images = np.zeros((n, size, size), dtype=np.float32)
    labels = rng.integers(0, len(kinds), n)
    for i, lab in enumerate(labels):
        r = rng.uniform(5, 10)
        cx, cy = rng.uniform(r + 1, size - r - 1, 2)
        dx, dy = xx - cx, yy - cy
        kind = kinds[lab]
        if kind == "circle":
            mask = dx ** 2 + dy ** 2 <= r ** 2
        elif kind == "square":
            mask = (abs(dx) <= r * 0.85) & (abs(dy) <= r * 0.85)
        elif kind == "triangle":
            mask = (dy >= -r) & (dy <= r) & (abs(dx) <= (dy + r) / 2)
        elif kind == "ring":
            mask = (dx ** 2 + dy ** 2 <= r ** 2) & (dx ** 2 + dy ** 2 >= (0.55 * r) ** 2)
        elif kind == "diamond":
            mask = abs(dx) + abs(dy) <= r
        else:  # cross
            mask = ((abs(dx) <= 0.3 * r) & (abs(dy) <= r)) | ((abs(dy) <= 0.3 * r) & (abs(dx)
<= r))
        images[i][mask] = rng.uniform(0.6, 1.0)
        images[i] += rng.normal(0, 0.15, (size, size))
    images = torch.tensor(np.clip(images, 0, 1)).unsqueeze(1)   # (n, 1 channel, 32, 32)
    return images, torch.tensor(labels)

X_all, y_all = make_shapes(3000)
X_train, y_train = X_all[:2400], y_all[:2400]
X_test, y_test = X_all[2400:], y_all[2400:]
names = ["circle", "square", "triangle"]

fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for ax, img, lab in zip(axes, X_train, y_train):
    ax.imshow(img[0], cmap="gray")
    ax.set_title(names[lab])
    ax.axis("off")
plt.show()
print(X_train.shape, torch.bincount(y_train))
```

The CNN

```
class ShapeCNN(nn.Module):
    def __init__(self, n_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 32 ->
16
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 16 -> 8
            nn.Flatten())
        self.head = nn.Sequential(nn.Linear(32 * 8 * 8, 64), nn.ReLU(), nn.Linear(64,
n_classes))

    def forward(self, x):
        return self.head(self.features(x))

def train_model(model, X, y, epochs=5, batch=64, lr=1e-3, params=None, augment=None):
    opt = torch.optim.Adam(params if params is not None else model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(X))
        for start in range(0, len(perm), batch):
            idx = perm[start:start + batch]
            xb = X[idx]
            if augment is not None:
                xb = torch.stack([augment(img) for img in xb])
            opt.zero_grad()
            F.cross_entropy(model(xb), y[idx]).backward()
            opt.step()
    return model

def accuracy(model, X, y):
    model.eval()
    with torch.no_grad():
        return round((model(X).argmax(dim=1) == y).float().mean().item(), 3)
```

The classic CNN pattern is **conv → ReLU → pool**, repeated, then **flatten** and a small classifier ("head").

Follow the shapes

```
model = ShapeCNN()
x = X_train[:4]
print("input:          ", tuple(x.shape))
for layer in model.features:
    x = layer(x)
    print(f"{layer.__class__.__name__:15s}", tuple(x.shape))
print("output logits:  ", tuple(model.head(x).shape))
print("parameters:", sum(p.numel() for p in model.parameters()))
```

- After the first conv: 16 feature maps of 32 × 32.
- After pooling: 16 × 16 × 16. After the second conv and pool: 32 × 8 × 8.
- Flatten gives 32 × 8 × 8 = **2,048** numbers, and the head turns them into 3 logits.

Getting the number in `nn.Linear(32 * 8 * 8, 64)` right is the most common CNN bug — printing the shapes like this finds it in seconds.

Train it

```
torch.manual_seed(0)
cnn = train_model(ShapeCNN(), X_train, y_train, epochs=5)
print("CNN test accuracy:", accuracy(cnn, X_test, y_test))
```

After only 5 epochs the CNN recognises about **95%** of the test shapes.

Compare with a plain network

```
torch.manual_seed(0)
mlp = nn.Sequential(nn.Flatten(), nn.Linear(32 * 32, 64), nn.ReLU(), nn.Linear(64, 3))
mlp = train_model(mlp, X_train, y_train, epochs=5)
print("Plain network test accuracy:", accuracy(mlp, X_test, y_test))
print("plain network parameters:", sum(p.numel() for p in mlp.parameters()))
```

The plain network gets only about **0.67**. It sees 1,024 unconnected numbers and must learn each shape separately at every position. The CNN's filters detect edges and corners **wherever** the shape is — that is the power of convolution.

Look inside: the learned filters

```
filters = cnn.features[0].weight.detach()      # 16 filters of 1 x 3 x 3
fig, axes = plt.subplots(2, 8, figsize=(10, 3))
for ax, f in zip(axes.ravel(), filters):
    ax.imshow(f[0], cmap="bwr")
    ax.axis("off")
plt.suptitle("First-layer filters learned from the data")
plt.show()

maps = F.relu(cnn.features[0](X_test[:1])).detach()[0]
fig, axes = plt.subplots(2, 8, figsize=(10, 3))
for ax, m in zip(axes.ravel(), maps):
    ax.imshow(m, cmap="gray")
    ax.axis("off")
plt.suptitle("Feature maps for one test image")
plt.show()
```

Many filters look like edge detectors in different directions — the network discovered them on its own.

Famous CNN architectures

Network	Year	Idea
LeNet	1998	Read handwritten cheques and postcodes
AlexNet	2012	Won ImageNet by a huge margin on GPUs — started the deep-learning boom
VGG	2014	Many small 3 × 3 convolutions
ResNet	2015	Residual connections; 50–150 layers
EfficientNet, ConvNeXt	2019–2022	Better accuracy per computation

Today, **vision transformers** compete with CNNs on large datasets, but CNNs remain fast, reliable and excellent for phones and small datasets.

Key terms

- **CNN** — a network built from convolution layers for images.
- **conv → ReLU → pool** — the standard CNN building block.
- **Flatten** — turn feature maps into one long vector for the classifier head.
- **Head** — the final layers that turn features into class scores.
- **Architecture** — the arrangement of layers in a network.

Remember this: A CNN stacks conv–ReLU–pool blocks that learn reusable visual patterns, so it beats a plain network on images with fewer, smarter weights.

Name: _____ Date: _____

Activity

Task 1

Add a third conv block (32 → 64 filters). What must change in the head?

Task 2

Train for 10 epochs. How much better does the CNN get?

Task 3

Replace max pooling with `stride=2` in the convolutions. Does accuracy change?

Task 4

Find three test images the CNN gets wrong and show them. What makes them hard?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is the classic building block of a CNN?
   - A. Conv → ReLU → pool
   - B. Softmax → dropout → softmax
   - C. Embedding → LSTM
   - D. Sort → split → join
2. After two 2 × 2 poolings, what size is a 32 × 32 image?
   - A. 32 × 32
   - B. 8 × 8
   - C. 16 × 16
   - D. 4 × 4
3. With 32 channels of 8 × 8, how many numbers come out of Flatten?
   - A. 2,048
   - B. 64
   - C. 32
   - D. 256
4. Why did the CNN beat the plain network on the shapes?
   - A. It had more data
   - B. Plain networks cannot use ReLU
   - C. It used a different test set
   - D. Its filters detect patterns wherever the shape appears
5. What is a quick way to find a size mismatch bug in a CNN?
   - A. Pass a small batch through the layers and print each output shape
   - B. Delete the head
   - C. Retrain from scratch
   - D. Use a bigger learning rate
6. Which 2015 architecture introduced residual connections?
   - A. VGG
   - B. LeNet
   - C. ResNet
   - D. AlexNet

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.